Portfolio Intelligence Platform

Technical Documentation

Generated on 2026-07-08

1. Executive Summary

Portfolio Intelligence Platform is a Vite + React + TypeScript application for analyzing mutual fund portfolio statements, scoring each fund against an appropriate benchmark, and producing investor-ready PDF reports. The product has two user-facing surfaces:

A public client experience for uploading a PDF statement, extracting fund data, and receiving a portfolio review.

An admin console for benchmark management, category-to-benchmark mapping, report generation, and historical report browsing.

The implementation combines browser-side PDF parsing, a serverless Supabase Edge Function, deterministic alpha-scoring logic, and a Supabase Postgres data layer. The product is designed to be privacy-aware: PDFs are parsed locally in the browser, and the raw statement content is not stored persistently.

2. Product Goals and Scope

The platform is intended to support the following workflows:

1. Upload a portfolio PDF statement. 2. Extract fund rows from the statement. 3. Map each fund to the most relevant benchmark index. 4. Compute benchmark-relative performance using age-adjusted XIRR logic. 5. Classify each fund into STAR, GOOD, REVIEW, or EXIT. 6. Produce a branded PDF report for investor sharing or advisor review. 7. Let admins manage benchmark data and category mapping rules without changing code.

The system is built around the idea of “alpha” as a benchmark-relative measure rather than a generic fund return metric.

3. Architecture Overview

3.1 High-Level Components

Frontend shell: React 19 application bootstrapped with Vite 8.

Client experience: landing page, upload flow, analysis progress UI, and results view.

Admin console: protected routes for managing benchmark data, category mappings, and reports.

Shared domain logic: alpha scoring engine, PDF text extraction, PDF report generation, auth/profile helpers, and Supabase client wrapper.

Serverless analysis service: Deno-based Edge Function that calls Anthropic for fund extraction and persists a report history row.

Data layer: Supabase Postgres with Row Level Security (RLS) and migration-managed schema.

3.2 Runtime Flow

1. The client uploads a PDF in the browser. 2. The browser extracts text using PDF.js. 3. The extracted text is sent to the Supabase Edge Function. 4. The Edge Function authenticates the caller, enforces a daily report limit for clients, calls Anthropic, and parses the AI output into fund records. 5. The frontend applies the shared alpha engine to transform raw fund records into scored FundRecord objects. 6. The results are shown on screen and optionally exported into a PDF. 7. Admin workflows use the same analysis pipeline, with an admin-only token and separate UI controls.

4. Technology Stack

4.1 Frontend

React 19

TypeScript

Vite 8

Tailwind CSS 4

Framer Motion for transitions and reveal animations

React Router for client/admin routing

Radix UI primitives for accessible UI surfaces

Lucide React for icons

Recharts, xlsx, jsPDF, pdfjs-dist, react-dropzone

4.2 Backend and Services

Supabase for auth, database, and Edge Functions

Deno runtime for the analysis service

Anthropic Claude API for PDF text-to-fund extraction

Optional Nirixa tracking for analytics/observability

4.3 Data and Storage

Supabase Postgres

Row Level Security policies

JSONB storage for fund payloads and report metadata

5. Repository Structure

The application is organized around clear domain areas:

src/App.tsx: top-level router and lazy-loaded routes.

src/client/: public client portal and landing experience.

src/admin/: admin console routes and pages.

src/shared/: reusable logic and domain services.

src/components/ui/: shared UI primitives built on Radix UI.

supabase/functions/analyse/: serverless analysis endpoint.

supabase/migrations/: versioned database schema evolution.

5.1 Main Entry Points

src/main.tsx: application bootstrap.

src/App.tsx: route structure and suspense boundaries.

src/client/ClientApp.tsx: main client analysis workflow.

src/admin/AdminApp.tsx: protected admin routing.

5.2 Shared Modules

src/shared/alphaEngine.ts: benchmark selection, XIRR calculation, signal classification, and benchmark series loading.

src/shared/pdfExtract.ts: PDF text extraction using pdfjs-dist.

src/shared/pdfReport.ts: branded PDF generation using jsPDF.

src/shared/types.ts: shared domain types and default thresholds.

src/shared/supabaseClient.ts: singleton Supabase client initialization.

src/shared/profile.ts: profile loading and profile refresh hooks.

src/shared/hooks/useAuth.ts: authentication state management.

6. Client Experience

6.1 Landing Page Composition

The public route renders a marketing-style landing experience built from section components:

Hero

LogoCloud

AnalyzeSection

HowItWorks

Features

SampleInsights

Testimonials

Trust

FAQ

CtaStrip

Footer

The experience is designed to feel like an advisor-led review rather than a generic upload form.

6.2 Analysis Workflow

The main analysis flow is implemented in src/client/ClientApp.tsx and follows these steps:

1. The user selects a PDF. 2. The app checks authentication state. 3. If not signed in, the app opens a login modal and stores an “analyze intent” in session storage. 4. Once authenticated, the user can trigger analysis. 5. The PDF is parsed locally in the browser. 6. A POST request is sent to the Edge Function with the extracted text and authorization details. 7. The returned funds are processed by the alpha engine. 8. The resulting fund list is displayed with investor name and signal summary. 9. The report row is patched with average alpha, average XIRR, and signal counts.

6.3 Client-Side UX Features

Smooth section scroll

Login modal integration with OAuth redirect recovery

Toast notifications for auth failures and completion states

Rate-limit messaging after the daily free-report allotment is exhausted

Progressive feedback during PDF extraction and analysis

7. Admin Experience

7.1 Protected Admin Console

The admin console is lazily loaded and guarded by profile-based auth checks. Access to /admin/* requires:

a valid Supabase session

a profile row with is_admin = true

The entry point in src/admin/AdminApp.tsx redirects clients and unauthenticated visitors appropriately.

7.2 Admin Pages

Dashboard: high-level admin landing experience.

BenchmarkManager: import, update, clear, and add benchmark price series.

CategoryMapping: manage category-to-benchmark mappings and keyword rules.

GenerateReport: upload a PDF, generate an analysis, adjust thresholds, and export PDF.

ReportHistory: review all reports with filtering and re-download capability.

7.3 Admin-Specific Analysis Path

The admin report generation page uses a special header, x-admin-token: true, to bypass the normal client-only daily limit and send content directly to the analysis service.

8. Shared Business Logic

8.1 PDF Extraction

The module src/shared/pdfExtract.ts uses pdfjs-dist to extract text from uploaded PDFs. The implementation:

loads the worker on demand

parses every page

sorts text items top-to-bottom and left-to-right to preserve approximate layout ordering

concatenates the result into a single text string for the analysis service

8.2 Alpha Engine

The core domain logic lives in src/shared/alphaEngine.ts. It is responsible for:

parsing flexible date formats from statement data and benchmark files

computing XIRR via Newton–Raphson with bisection fallback

loading benchmark price series from Supabase

load and cache category mapping rules from Supabase

selecting a benchmark index based on category and keyword rules

computing benchmark XIRR from the fund start date to the latest benchmark date

computing alpha = fund XIRR - benchmark XIRR

classifying signals using threshold rules

8.3 Signal Classification

Signal logic is intentionally simple and deterministic:

Funds younger than the configured age threshold are marked REVIEW.

Funds with alpha below the exit threshold are marked EXIT.

Funds with negative alpha but older than the threshold are marked REVIEW.

Funds with alpha below the star threshold are marked GOOD.

Everything else is STAR.

8.4 PDF Report Generation

The module src/shared/pdfReport.ts renders a branded PDF using jsPDF. It includes:

a title block and report date

investor summary metrics

tier-based scorecards

fund analysis tables

EXIT and REVIEW detail sections

support for multi-investor family reports

This functionality is used in the client results experience and in the admin-generated report flow.

9. Analysis Edge Function

The serverless endpoint lives at supabase/functions/analyse/index.ts.

9.1 Responsibilities

Accept PDF text from the browser.

Validate authentication.

Enforce a per-user daily limit of three reports for client traffic.

Build a prompt for Anthropic Claude.

Send the request to Anthropic.

Parse Claude’s JSON response into fund row objects.

Insert a report_history row for persistence.

Return the extracted funds and report identifier to the frontend.

9.2 Security and Input Handling

The function uses the Supabase service role key internally for persistence.

The endpoint supports CORS headers for browser clients.

Admin-generated requests bypass the normal client auth limit using x-admin-token.

The function is deliberately strict about the expected JSON structure from Claude.

9.3 Failure Modes

Invalid JSON from Claude returns a 500 response.

Anthropic failures return a 502-style error payload from the Edge Function.

Missing or too-short PDF text returns a 400 error.

A client hitting the daily limit receives a 429 response.

10. Data Model

10.1 Core Tables

profiles: stores profile metadata and admin status for auth users.

report_history: stores each analysis run, investor name, fund count, and raw funds payload.

benchmark_prices: stores benchmark TRI price series by index/date.

category_mappings: stores category rules mapping a category string to a benchmark and keyword list.

10.2 Key Relationships

profiles.id references auth.users.id.

report_history.user_id references auth.users.id and is used for per-user history and RLS.

benchmark_prices is keyed by index_code + date.

category_mappings is keyed by category and stores the logic used by the alpha engine.

10.3 Schema Evolution

The migration chain shows an evolution from a precomputed benchmark lookup table towards a runtime benchmark-series model:

Initial schema used benchmark_data and category_mappings.

Later migrations introduced benchmark_prices and widened benchmark support.

Additional migrations added profile and report_history support for auth-aware history.

11. Authentication and Authorization

11.1 Client Auth

The client uses Supabase auth through the shared hook in src/shared/hooks/useAuth.ts and the Supabase client wrapper in src/shared/supabaseClient.ts.

11.2 Admin Access Control

Admin access is not based on a hard-coded list. Instead, it is driven by the profiles table and the is_admin flag.

11.3 Row Level Security

Supabase RLS governs what authenticated users can read or update:

Users can view their own profile data.

Users can view their own report_history rows.

Users can update their own report_history rows after the edge function inserts the initial row.

Admins can read the full report history.

This makes the app safer than a purely client-side model while keeping the implementation straightforward.

12. Security and Privacy Considerations

The product’s privacy story is one of its strongest design points.

PDF parsing happens in the browser before any network call.

The uploaded statement is not persisted by the frontend.

The Edge Function receives extracted text, not the full source PDF.

The analysis call is transient and not used to train the product.

Supabase RLS and auth gating limit who can read reports or access admin features.

Potential security considerations going forward:

The serverless function currently trusts the provided token and admin header context; it should continue to be monitored for abuse.

The product should maintain careful handling of environment variables and service keys.

Any future data export features should keep the same “minimum necessary data” posture.

13. Configuration and Environment Variables

13.1 Frontend Variables

VITE_SUPABASE_URL

VITE_SUPABASE_ANON_KEY

13.2 Edge Function Variables

SUPABASE_URL

SUPABASE_SERVICE_ROLE_KEY

ANTHROPIC_API_KEY

NIRIXA_API_KEY (optional)

The frontend config is loaded from import.meta.env and the edge service reads environment variables from the Deno runtime.

14. Build, Development, and Deployment

14.1 Local Development

From the project root:

npm install

npm run dev

14.2 Production Build

npm run build

The Vite configuration includes path aliases and a dependency optimization step for pdfjs-dist.

14.3 Hosting Model

The application is designed for a modern frontend hosting environment with a Supabase backend and Edge Functions. The repository also includes a vercel.json file, indicating compatibility with Vercel-style hosting patterns.

15. Testing and Reliability Notes

The codebase currently emphasizes implementation correctness and user experience over exhaustive automated testing. The main resilience mechanisms in place are:

explicit error handling in the client and edge service

cache invalidation after benchmark or mapping changes

fallback benchmark values when benchmark data is missing

progress feedback during long-running analysis steps

The system would benefit from:

unit tests for the alpha engine

integration tests for the edge function payload parsing

UI tests for the upload and auth flows

regression tests for report PDF generation

16. Key Strengths

Clear separation between UI, domain logic, and data services

Deterministic alpha scoring grounded in benchmark series and category mapping

Strong admin controls for benchmark and mapping maintenance

Privacy-aware PDF handling

Reusable PDF report generation for both client and admin workflows

17. Risks and Recommendations

17.1 Current Risks

The product depends on AI extraction quality for fund rows; bad OCR or prompt drift can affect accuracy.

The alpha engine currently relies on benchmark series availability and mapping quality.

The client history and report workflow depend heavily on the structure of the raw extracted fund rows.

17.2 Recommended Next Steps

1. Add automated tests around the alpha engine and edge-function JSON parsing. 2. Add structured logging and telemetry around analysis failures and rate-limit events. 3. Improve document parsing robustness with fallback prompts or OCR preprocessing. 4. Add a dedicated admin dashboard for usage trends and daily report counts. 5. Introduce versioned benchmark import validation to detect malformed or incomplete series.

18. Summary

Portfolio Intelligence Platform is a full-stack portfolio analysis product that brings together PDF parsing, AI extraction, benchmark-relative performance scoring, and investor-ready reporting in one user experience. Its architecture balances client-side interactivity with server-side intelligence, and it is structured to support both retail investors and advisors through a shared analytical engine.